

MATH00041: Introduction to Programming

Object-Oriented Programming

Lecture 7

Archontis Giannakidis, PhD

Outline

- **Class**
- **Objects**
- **Inheritance**
- **Polymorphism**
- **Encapsulation**
- **Data Abstraction**

Python OOPs Concepts

Python supports Object-Oriented Programming (OOP), which is a programming paradigm that uses objects and classes to represent real-world entities like inheritance, polymorphism, encapsulation, etc. The main idea behind OOP is to bind data and the functions that operate on it together as a single entity so that no other part of the code can access this data. This approach allows for better code organization, reusability, and scalability.

OOPs Concepts in Python

- Class
- Objects
- Inheritance
- Polymorphism
- Encapsulation
- Data Abstraction



Python Classes and Objects

Python is a programming language that uses the concept of objects. In Python, everything is considered as an object with its unique characteristics and functions. A Class is a blueprint for creating objects in Python. It defines the characteristics and functions of an object. When an object is created based on a Class, it inherits all the properties and methods defined in the Class.

Python Class

A class is a collection of objects. A class contains the blueprints or the prototype from which the objects are being created. It is a logical entity that contains some attributes and methods.

To understand the need for creating a class, let's consider an example, let's say you wanted to track the number of dogs that may have different attributes like breed, and age. If a list is used, the first element could be the dog's breed, while the second element could represent its age. Let's suppose there are 100 different dogs, then how would you know which element is supposed to be which? What if you wanted to add other properties to these dogs? This lacks organisation, and it's the exact need for classes.

Some points on Python class:

- Classes are created by keyword class.
- Attributes are the variables that belong to a class.
- Attributes are always public and can be accessed using the dot (.) operator.

Eg.: Myclass.Myattribute

Python Objects

The object is an entity that has a state and behaviour associated with it. It may be any real-world object like a mouse, keyboard, chair, table, pen, etc. Integers, strings, floating-point numbers, even arrays, and dictionaries, are all objects. More specifically, any single integer or any single string is an object. The number 12 is an object, the string “Hello, world” is an object, a list is an object that can hold other objects, and so on. You’ve been using objects all along and may not even realise it.

An object consists of:

- **State:** It is represented by the attributes of an object. It also reflects the properties of an object.
- **Behavior:** It is represented by the methods of an object. It also reflects the response of an object to other objects.
- **Identity:** It gives a unique name to an object and enables one object to interact with other objects.

To understand the state, behaviour, and identity, let us take the example of the class dog (explained above).

- 1) The identity can be considered as the name of the dog.
- 2) State or Attributes can be considered as the breed, age, or colour of the dog.
- 3) The behaviour can be considered as to whether the dog is eating or sleeping.

Create a Class

To create a class, use the keyword **class**:

Example:

Create a class named MyClass, with a property named x:

```
class MyClass:  
    x = 5  
print(MyClass)
```

Output: <class '__main__.MyClass'>

Create Object

Now we can use the class named MyClass to create objects:

Example:

Create an object named p1, and print the value of x:

```
p1 = MyClass()  
print(p1.x)
```

Output: 5

The `__init__()` Function

Classes and objects are fundamental concepts in object-oriented programming. However, the examples provided earlier may not be practical in real-life applications as they are very basic.

To understand the concept of classes, it is essential to know about the `__init__()` function. This function is present in all classes and is executed every time a class is initiated. The `__init__()` function is used to assign values to object properties and perform other necessary operations while creating an object.

In simpler terms, you can think of the `__init__()` function as a constructor method that initializes the state of an object by setting its properties and performing any other necessary operations. By using the `__init__()` function, you can ensure that the object is created with all the required values, making it easier to use and manipulate in your code.

Example:

Create a class named Person, use the `__init__()` function to assign values for name and age:

```
class Person:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
p1 = Person("John", 36)
```

```
print(p1.name)  
print(p1.age)
```

Output: John

Output: 36

- ❖ **Note:** The `__init__()` function is called automatically every time the class is being used to create a new object.

The `__str__()` Function

In Python, the `__str__()` function is used to define the string representation of an object of a class. When an object is printed or converted to a string, this function is called to produce the output. If the `__str__()` function is not defined for a class, the default string representation of the object is returned. Therefore, defining the `__str__()` function is an important aspect of creating a class in Python, as it allows for custom string representations of objects, which can be more informative and easier to read than the default representation.

Example:

The string representation of an object WITHOUT the `__str__()` function:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("John", 36)
```

```
print(p1)
```

Output: `<__main__.Person object at 0x15039e602100>`

Example:

The string representation of an object WITH the `__str__()` function:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}({self.age})"

p1 = Person("John", 36)

print(p1)
```

Output: John(36)

Object Methods

In object-oriented programming, objects can have functions associated with them, which are called methods. These methods are defined within the object's class and can access and modify the object's data. This allows for more organized and modular code, as related functions can be grouped together within a single object.

Example:

Insert a function that prints a greeting, and execute it on the p1 object:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.myfunc()
```

Output: Hello my name is John

- ❖ **Note:** The self parameter is a reference to the current instance of the class and is used to access variables that belong to the class.

The self Parameter

In object-oriented programming, a class is a blueprint for creating objects that share similar attributes and methods. When you create an instance of a class, you are creating a unique object that has its own set of instance variables and methods.

The `"self"` parameter is used within methods of a class to refer to the instance of the class that the method is being called on. It allows you to access and modify the instance variables of that object.

While `"self"` can technically be named anything, it's a convention in Python to use `"self"` as the parameter name. Additionally, it's important to note that `"self"` must always be the first parameter of any method within a class definition. This is because when an instance method is called on an object, Python automatically passes in the instance as the first argument (i.e. the `"self"` parameter).

Example:

Use the words *mysillyobject* and *abc* instead of *self*:

```
class Person:
    def __init__(mysillyobject, name, age):
        mysillyobject.name = name
        mysillyobject.age = age

    def myfunc(abc):
        print("Hello my name is " + abc.name)

p1 = Person("John", 36)
p1.myfunc()
```

Output: Hello my name is John

Modify Object Properties

In Python, you can modify the properties of an object by directly accessing its attributes and assigning them new values. To modify the value of a property, simply assign a new value to the corresponding attribute of the object.

Example:

Set the age of p1 to 40:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.age = 40
print (p1.age)
```

Output: 40

Delete Object Properties

In Python, objects are like containers that can store values or attributes. Each attribute has a name and a value. To delete an attribute from an object, you can use the `del` keyword followed by the name of the attribute that you want to delete. For example, if you have an object `my_object` with an attribute `my_attribute`, you can delete the attribute using `del my_object.my_attribute`.

Example:

Delete the age property from the p1 object.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

p1 = Person("John", 36)
del p1.age
print (p1.age)
```

Output:

Traceback (most recent call last):

File "demo_class7.py", line 13, in <module>

print(p1.age)

AttributeError: 'Person' object has no attribute 'age'

The pass Statement

In Python programming language, a `class` definition cannot be empty. This means that there has to be some content within the `class` definition block. However, if for some reasons you do have an empty `class` definition, it is recommended to add the `'pass'` statement to avoid getting an error. The `'pass'` statement is used as a placeholder to indicate that there is no code to be executed and it allows an empty block of code to be valid in Python.

Example:

```
class Person:  
    pass
```

Output: # having an empty class definition like this, would raise an error without the pass statement.

Python Inheritance

Inheritance is a concept in programming where we can create a new class (**child class**) based on an existing class (**parent class**). This allows us to reuse the code written in the parent class and add new functionality to the child class without having to write the code again.

The parent class is also known as the base class because it provides the basic blueprint for the child class. The child class, on the other hand, is also known as the derived class because it derives its properties and behaviours from the parent class.

Think of it like a family tree. The parent class is like the parent in a family, and the child class is like the child. The child inherits certain traits and characteristics from the parent but can also have its own unique traits.

Inheritance is an important concept in object-oriented programming and is used extensively in many programming languages.

- **The parent class** is the class being inherited from, also called the base class.
- **The Child class** is the class that inherits from another class, also called derived class.

Types of Inheritance

- **Single Inheritance:** Single-level inheritance enables a derived class to inherit characteristics from a single-parent class.
- **Multilevel Inheritance:** Multi-level inheritance enables a derived class to inherit properties from an immediate parent class, which in turn inherits properties from his parent class.
- **Hierarchical Inheritance:** Hierarchical-level inheritance enables more than one derived class to inherit properties from a parent class.
- **Multiple Inheritance:** Multiple-level inheritance enables one derived class to inherit properties from more than one base class.

Create a Parent Class

When you create a parent class, the syntax is no different from creating any other class. This means that any class can be a parent class.

Example:

Create a class named `Person`, with `firstname` and `lastname` properties, and `printname` method:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)
```

#Use the `Person` class to create an object, and then execute the `printname` method:

```
x = Person("John", "Doe")
x.printname()
```

Output: John Doe

Create a Child Class

When creating a new class that needs to have the functionality of an existing class, you can create a child class that inherits all of the attributes and methods of the parent class by passing the parent class as a parameter during the creation of the child class. This allows you to reuse code and avoid duplication, making your code more efficient and maintainable.

Example:

Create a class named `Student`, which will inherit the properties and methods from the `Person` class:

```
class Student(Person):  
    pass
```

- ❖ **Note:** Use the `pass` keyword when you do not want to add any other properties or methods to the class.

Now, the Student class has the same properties and methods as the Person class.

Example:

Use the `Student` class to create an object, and then execute the `printname` method:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

x = Student("Mike", "Olsen")
x.printname()
```

Output: Mike Olsen

Add the `__init__()` Function

So far, we have created a child class that inherits properties and methods from its parent. To replace the `pass` keyword, we can add the `__init__()` function to the child class.

- ❖ **Note:** The `__init__()` function is called automatically every time the class is being used to create a new object.

Example:

Add the `__init__()` function to the `Student` class:

```
class Student(Person):  
    def __init__(self, fname, lname):  
        #add properties etc.
```

When you add the `__init__()` function, the child class will no longer inherit the parent's `__init__()` function.

- ❖ **Note:** The child's `__init__()` function overrides the inheritance of the parent's `__init__()` function.

To keep the inheritance of the parent's `__init__()` function, add a call to the parent's `__init__()` function:

Example:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

class Student(Person):
    def __init__(self, fname, lname):
        Person.__init__(self, fname, lname)

x = Student("Mike", "Olsen")
x.printname()
```

Output: Mike Olsen

Now we have successfully added the `__init__()` function, and kept the inheritance of the parent class, and we are ready to add functionality in the `__init__()` function.

super() Function

In object-oriented programming, inheritance is a mechanism that allows a new class to be based on an existing class, inheriting all the properties and methods of the parent class. In Python, the `super()` function is used to refer to the parent class and can be used to call its methods. By using the `super()` function in the child class, you can inherit all the methods and properties of the parent class. This can save time by reducing the amount of code that you need to write and helps to keep your code organised.

Example:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

class Student(Person):
    def __init__(self, fname, lname):
        super().__init__(fname, lname)

x = Student("Mike", "Olsen")
x.printname()
```

Output: Mike Olsen

When using the `super()` function, you don't need to specify the name of the parent element. It automatically inherits methods and properties.

Add Properties

Example:

Add a property called **graduationyear** to the **Student** class:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

class Student(Person):
    def __init__(self, fname, lname):
        super().__init__(fname, lname)
        self.graduationyear = 2019

x = Student("Mike", "Olsen")
print(x.graduationyear)
```

Output: 2019

In the example below, the year **2019** should be a variable, and passed into the **Student** class when creating student objects. To do so, add another parameter in the `__init__()` function:

Example:

Add a **year** parameter, and pass the correct year when creating objects:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

class Student(Person):
    def __init__(self, fname, lname, year):
        super().__init__(fname, lname)
        self.graduationyear = year

x = Student("Mike", "Olsen", 2019 )
print(x.graduationyear)
```

Output: 2019

Add Methods

Example:

Add a method called `welcome` to the `Student` class:

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

class Student(Person):
    def __init__(self, fname, lname, year):
        super().__init__(fname, lname)
        self.graduationyear = year
    def welcome(self):
        print("Welcome", self.firstname, self.lastname, "to the class of", self.graduationyear)

x = Student("Mike", "Olsen", 2019 )
print(x.welcome() )
```

Output: Welcome Mike Olsen to the class of 2019

Python Polymorphism

Polymorphism is a concept in object-oriented programming that allows an object to take on different forms or behaviours. In simpler terms, it means that a single object can be used to perform different tasks or operations depending on the context in which it is used.

In summary, polymorphism allows for greater flexibility and reusability in programming by making it possible to use a single code block to handle multiple scenarios.

Class Polymorphism

Polymorphism allows multiple classes to have the same method name, commonly used in class methods. For instance, **Car**, **Boat**, and **Plane** can all have a **move()** method.

Example:

Different classes with the same method:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def move(self):
        print("Drive!")

class Boat:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def move(self):
        print("Sail!")
```

```
class Plane:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def move(self):
        print("Fly!")

car1 = Car("Ford", "Mustang")           #Create a Car class
boat1 = Boat("Ibiza", "Touring 20")    #Create a Boat class
plane1 = Plane("Boeing", "747")        #Create a Plane class

for x in (car1, boat1, plane1):
    x.move()
```

Output: Drive!

Output: Sail!

Output: Fly!

Look at the for loop at the end. Because of polymorphism, we can execute the same method for all three classes.

Inheritance Class Polymorphism

Regarding the use of polymorphism in classes that have child methods with the same name, the answer is yes. For instance, if we create a parent class called "Vehicle" and define "Car", "Boat", and "Plane" as child classes of "Vehicle", the child classes will inherit the methods of "Vehicle". However, they can also override these methods if necessary.

Example:

Create a class called **Vehicle** and make **Car**, **Boat**, **Plane** child classes of **Vehicle**:

```
class Vehicle:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def move(self):
        print("Move!")

class Car(Vehicle):
    pass

class Boat(Vehicle):
    def move(self):
        print("Sail!")
```

```
class Plane(Vehicle):
    def move(self):
        print("Fly!")

car1 = Car("Ford", "Mustang") #Create a Car object
boat1 = Boat("Ibiza", "Touring 20") #Create a Boat object
plane1 = Plane("Boeing", "747") #Create a Plane object

for x in (car1, boat1, plane1):
    print(x.brand)
    print(x.model)
    x.move()
```

Output: Ford

Output: Mustang

Output: Move!

Output: Ibiza

Output: Touring 20

Output: Sail!

Output: Boeing

Output: 747

Output: Fly!

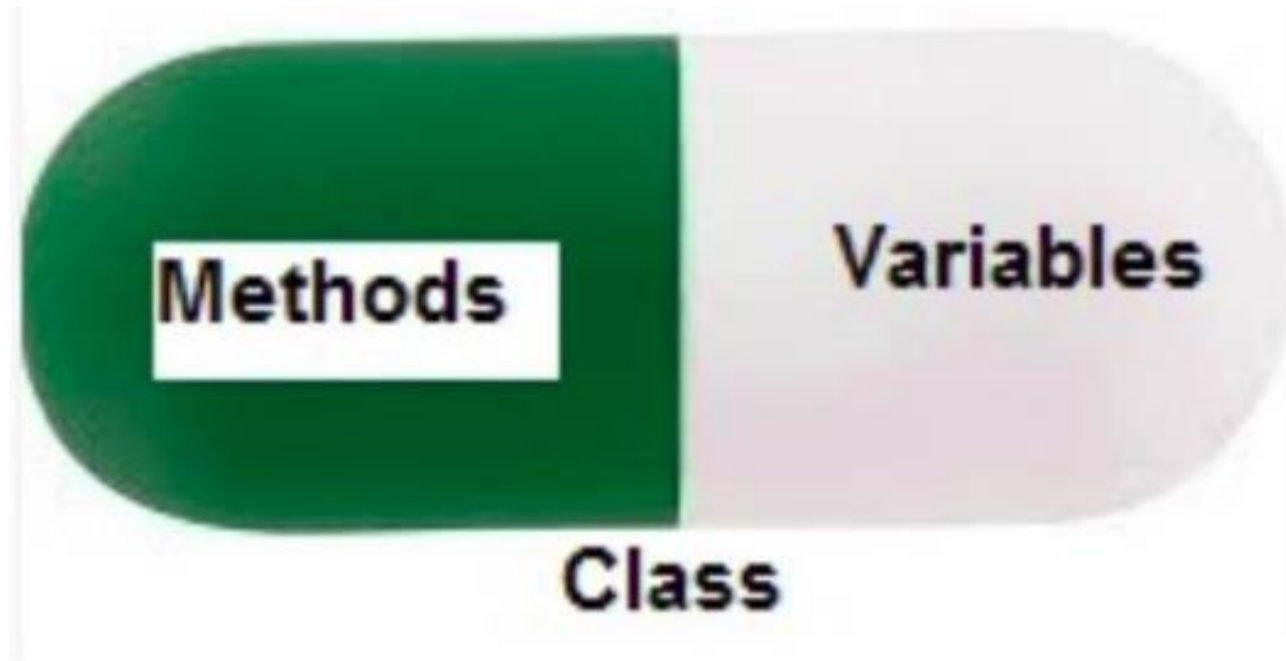
When a class is created, it can inherit properties and methods from an existing class, known as the parent class. This is called inheritance. For instance, in the example provided, the **Car** class inherits the properties of **brand**, **model**, and method **move()** from the **Vehicle** class, even though the **Car** class is empty. Similarly, the **Boat** and **Plane** classes inherit the properties of **brand**, **model**, and **move()** from the **Vehicle** class, but they override the **move()** method to make it specific to their respective classes.

Polymorphism allows us to execute the same method for all classes, making programming more efficient.

Python Encapsulation

Encapsulation is one of the fundamental concepts in object-oriented programming (OOP). It describes the idea of wrapping data and the methods that work on data within one unit. This puts restrictions on accessing variables and methods directly and can prevent the accidental modification of data. To prevent accidental change, an object's variable can only be changed by an object's method. Those types of variables are known as private variables.

A class is an example of encapsulation as it encapsulates all the data that is member functions, variables, etc.



Encapsulation in Python

In the following example, we have created the c variable as the private attribute. We cannot even access this attribute directly and can't even change its value.

Example:

```
# Creating a Base class
class Base:
    def __init__(self):
        self.a = "GeeksforGeeks"
        self.__c = "GeeksforGeeks "

# Creating a derived class
class Derived(Base):
    def __init__(self):

        # Calling constructor of
        # Base class
        Base.__init__(self)
        print("Calling private member of base class: ")
        print(self.__c)

# Driver code
obj1 = Base()
print(obj1.a)
```

Output: GeeksforGeeks

Data Abstraction

Data abstraction is a concept in computer science that refers to the technique of hiding complex implementation details of a system from the user, while providing a simple and easy-to-use interface.

In programming, data abstraction allows developers to create abstract classes that define a set of methods and properties that can be used to interact with the system, without revealing the underlying implementation details. This makes the code more modular, flexible, and easier to maintain, as changes can be made to the underlying implementation without affecting the interface.

In Python, we use abstract classes to achieve data abstraction. An abstract class is a class that cannot be instantiated, but can be subclassed to provide some implementation details. The subclasses of an abstract class must provide an implementation for all the abstract methods defined in the parent class. This ensures that the abstract class provides a consistent interface that can be used by other parts of the code without worrying about the implementation details.

Thank you